

Feature Scaling — Standardization

■ What's Covered

1. Why Feature Scaling is Needed
2. What is Standardization?
3. The Z-Score Formula — Deep Dive
4. Step-by-Step Worked Example
5. Which Algorithms Need It (and Why)?
6. StandardScaler in Scikit-learn
7. Effect on Distribution
8. Outliers and Standardization
9. When NOT to Standardize
10. Standardization vs Other Scalers
11. Common Mistakes to Avoid
12. Summary Table

1. Why Do We Need Feature Scaling?

Raw datasets collected from the real world almost always contain numerical columns that live on wildly different scales. Left unhandled, this scale imbalance silently poisons many machine learning algorithms — not by throwing an error, but by quietly biasing the model towards features that happen to have larger numeric values.

1.1 Classic Illustration

Consider a dataset used to predict whether a person will buy a house:

Feature	Typical Range
Age	18 – 65 (years)
Annual Income	■ 2,00,000 – ■ 50,00,000
Number of Dependents	0 – 6 (count)
Distance from City Centre	1 – 80 (km)
Credit Score	300 – 900 (points)

The numeric magnitude of Annual Income (millions of rupees) dwarfs every other feature. An algorithm that computes distances or dot products will treat income as if it is exponentially more important — even if domain knowledge says otherwise. Feature Scaling fixes this.

1.2 What Does 'Scaling' Actually Mean?

Feature scaling transforms the values of every numerical column so they occupy a comparable numeric range, without destroying the information or relative ordering of values within each column.

- Does NOT change the shape of the distribution.
- Does NOT change the rank order of observations.
- DOES make gradient-based optimisers converge faster and distance-based algorithms fairer.

2. What is Standardization?

Definition: Standardization (also called Z-Score Normalization) transforms a feature so that it has a mean (μ) of 0 and a standard deviation (σ) of 1. The resulting values are called Z-scores.

After standardization every feature — regardless of its original unit or scale — is expressed as 'how many standard deviations away from its own mean is this value?' This places all features on a common, dimensionless yardstick.

2.1 Intuitive Analogy

Imagine two students scored in different exams. Student A scored 75/100 in Maths; Student B scored 680/800 in SAT. Who performed better relative to their class? You cannot compare raw scores directly. But converting both to Z-scores — 'how many standard deviations above or below the class average?' — makes comparison instant and fair. That is exactly what standardization does for ML features.

3. The Z-Score Formula — Full Breakdown

$$z = (x - \mu) \div \sigma$$

Symbol	Meaning
x	Original value of the data point for a given feature
μ (mu)	Mean (average) of that feature across the entire training set
σ (sigma)	Standard Deviation of that feature across the entire training set
z	Standardized value (Z-score) — output of the transformation

3.1 What the Two Parts Do

- $(x - \mu)$ — Centering: subtracts the mean so the distribution shifts its centre to 0. Values above the mean become positive; values below become negative.
- $\div \sigma$ — Scaling: divides by standard deviation so ± 1 unit in Z-score always means ± 1 standard deviation. This makes spread comparable across all features.

3.2 Properties of Standardized Data

- Mean of the transformed column = exactly 0
- Standard Deviation of the transformed column = exactly 1
- Min / Max are NOT fixed — they depend on how extreme outliers are
- Z-score of 0 → value equals the mean
- Z-score of +2 → value is 2 standard deviations above the mean
- Z-score of -1.5 → value is 1.5 standard deviations below the mean

Important: Standardization does NOT produce a normal (bell-curve) distribution. If your data was right-skewed before, it will still be right-skewed after. Only the centre and scale change — the shape is preserved.

4. Step-by-Step Worked Example

Manually standardizing 'Age' and 'Income' on a tiny dataset shows exactly what happens at each step.

4.1 Raw Data

Person	Age (years)	Income (■ lakh/yr)
A	25	4
B	30	8
C	35	6
D	40	12
E	45	10

4.2 Step 1 — Compute Mean (μ)

```
# Add all values and divide by count
mu_Age = (25 + 30 + 35 + 40 + 45) / 5
# = 175 / 5 = 35

mu_Income = (4 + 8 + 6 + 12 + 10) / 5
# = 40 / 5 = 8
```

Python

4.3 Step 2 — Compute Standard Deviation (σ)

Python

```
# Variance = average of squared deviations from mean
Var_Age = [(25-35)2 + (30-35)2 + (35-35)2 + (40-35)2 + (45-35)2] / 5
# = [100 + 25 + 0 + 25 + 100] / 5 = 250 / 5 = 50
sigma_Age = sqrt(50) ≈ 7.07

Var_Income = [(4-8)2 + (8-8)2 + (6-8)2 + (12-8)2 + (10-8)2] / 5
# = [16 + 0 + 4 + 16 + 4] / 5 = 40 / 5 = 8
sigma_Income = sqrt(8) ≈ 2.83
```

4.4 Step 3 — Apply $z = (x - \mu) / \sigma$

Person	Z_Age = (Age - 35) / 7.07	Z_Income = (Income - 8) / 2.83
A	(25-35)/7.07 ≈ -1.41	(4-8)/2.83 ≈ -1.41
B	(30-35)/7.07 ≈ -0.71	(8-8)/2.83 ≈ 0.00
C	(35-35)/7.07 ≈ 0.00	(6-8)/2.83 ≈ -0.71
D	(40-35)/7.07 ≈ +0.71	(12-8)/2.83 ≈ +1.41
E	(45-35)/7.07 ≈ +1.41	(10-8)/2.83 ≈ +0.71

Observation: Before scaling, Income ranged from 4 to 12 and Age from 25 to 45. After standardization both columns range from about -1.41 to +1.41 — perfectly comparable, with no loss of relative information.

4.5 Verify — Mean = 0, Std = 1

Python

```
# Quick sanity check on Z_Age column
mean_Z_Age = (-1.41 + -0.71 + 0.00 + 0.71 + 1.41) / 5
# ≈ 0.00 ■

std_Z_Age ≈ 1.00 ■
```

5. Which Algorithms Need Standardization — and Why?

Not all algorithms are affected by feature scale. Understanding the mechanism of each algorithm tells you immediately whether standardization is mandatory, optional, or irrelevant.

5.1 Must Standardize → Distance / Dot-Product / Gradient Algorithms

KNN (K-Nearest Neighbours)

- Classifies a new point by finding the K closest training points using Euclidean distance: $d = \sqrt{[\sum (x_i - y_i)^2]}$
- A feature with values in the thousands completely dominates the distance — smaller-scale features become invisible.
- After standardization: all features contribute proportionally.

SVM (Support Vector Machine)

- Finds a hyperplane that maximally separates classes using dot products between feature vectors.
- Large-scale features create a steep gradient in one direction, making the optimal hyperplane hard to find.
- Standardization gives the optimizer a balanced landscape.

PCA (Principal Component Analysis)

- Finds directions of maximum variance in the data.
- Without scaling, the largest-scale feature artificially dominates the first principal component.
- Rule: ALWAYS standardize before applying PCA.

Linear / Logistic Regression with Regularization

- Without regularization: scale-invariant (coefficients compensate).
- With L1 / L2 regularization: penalty treats all coefficients equally. Large-scale features get tiny coefficients — regularization penalizes them incorrectly.
- After standardization: regularization is fair and coefficient magnitudes reflect true feature importance.

Neural Networks / Deep Learning

- Gradient Descent updates weights proportionally to the gradient.
- Large-scale features → exploding gradients; tiny-scale features → vanishing gradients.
- Standardization gives the optimizer a smooth, balanced loss landscape → faster convergence, more stable training.

5.2 Scaling is Optional → Tree-Based Algorithms

Algorithm	Why Scale Doesn't Matter
Decision Trees	Splits on one feature using a threshold — other feature scales are irrelevant
Random Forests	Ensemble of decision trees — same reasoning applies
XGBoost / LightGBM	Tree-based boosting — splits use relative ordering, not absolute magnitude
Naïve Bayes	Probability computed per feature independently — scale doesn't affect ratio

Practical rule: If your algorithm uses distances, dot products, or gradient descent → standardize. If it uses feature thresholds (tree-based) → scaling is not required (but will not hurt).

6. StandardScaler in Scikit-learn

Scikit-learn's StandardScaler automates computing μ and σ and applies the z-score transformation efficiently.

6.1 Core API

```
# Import the scaler
from sklearn.preprocessing import StandardScaler

# Create a scaler instance
scaler = StandardScaler()

# Fit on training data – learns  $\mu$  and  $\sigma$  from X_train
scaler.fit(X_train)

# Transform training data using learned  $\mu$  and  $\sigma$ 
X_train_scaled = scaler.transform(X_train)

# Transform test data using the SAME  $\mu$  and  $\sigma$  (from training)
X_test_scaled = scaler.transform(X_test)

# Shortcut: fit and transform training data in one call
X_train_scaled = scaler.fit_transform(X_train)
```

Python

6.2 Inspecting Learned Parameters

```
# Inspect what the scaler learned from training data
print('Feature means ( $\mu$ ):', scaler.mean_)
print('Feature std devs ( $\sigma$ ):', scaler.scale_)
```

Python

6.3 Inverse Transform — Recover Original Scale

```
# Reverse the standardization – useful for interpreting predictions
X_original = scaler.inverse_transform(X_train_scaled)
```

Python

CRITICAL — Why fit ONLY on training data? The scaler must see ONLY the training set when computing μ and σ . Fitting on the entire dataset (including test data) means the model has indirectly 'seen' the test set — this is data leakage. It makes test accuracy artificially optimistic and the model underperforms in production.

6.4 Correct Train-Test Split Workflow

Python

```
# Step 1: Split FIRST – before any scaling
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Step 2: Fit scaler on TRAINING DATA ONLY
scaler = StandardScaler()
scaler.fit(X_train)

# Step 3: Transform BOTH splits using training statistics
X_train_sc = scaler.transform(X_train)
X_test_sc = scaler.transform(X_test) # Never fit on test!

# Step 4: Train model on scaled training data
model.fit(X_train_sc, y_train)

# Step 5: Evaluate on scaled test data
score = model.score(X_test_sc, y_test)
```

6.5 Pipeline — Best Practice (Eliminates Leakage Risk)

Scikit-learn Pipelines chain preprocessing and modelling steps. The pipeline automatically handles the fit/transform distinction correctly every time.

Python

```
# Import pipeline and components
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# Build pipeline: scaling → model
pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LogisticRegression())
])

# fit() calls scaler.fit_transform on X_train, then model.fit
pipe.fit(X_train, y_train)

# score() calls scaler.transform on X_test automatically
pipe.score(X_test, y_test)
```

Using a Pipeline completely eliminates the risk of accidentally fitting the scaler on test data. In production code, always use Pipelines.

7. Effect on Distribution — Visual Intuition

Understanding what standardization changes and what it preserves is crucial for interpreting your data correctly after transformation.

Property	Before Standardization	After Standardization
Mean (centre)	Any value (e.g., 35, 8000)	Always exactly 0
Std Deviation	Any value (e.g., 7.07, 2.83)	Always exactly 1
Distribution shape	Normal / Skewed / Bimodal — anything	Exactly same shape — UNCHANGED
Outlier influence	Outliers in original units	Outliers still exist as extreme Z-scores
Relative order	$A > B$ in original data	$A > B$ in standardized data — preserved
Unit	Original unit (kg, ■, years...)	Dimensionless (standard deviations)
Min / Max	Bounded by data range	Not bounded — depends on outliers

Key Insight: Standardization is a linear transformation — it only shifts and rescales. It CANNOT fix a skewed distribution. If you need to make data more symmetric, apply a log transform or Box-Cox transform BEFORE standardization.

8. How Outliers Affect Standardization

Standard deviation is itself sensitive to outliers. One extreme value can pull σ far higher than it should be, compressing all normal values close to zero.

8.1 Worked Example

```
# Dataset with one outlier age value
Ages = [25, 28, 30, 32, 35, 200] # 200 is a data error

# Mean is pulled high by the outlier
mu = (25+28+30+32+35+200) / 6 = 58.3

# Std dev is enormously inflated
sigma = 64.8

# Normal ages all compress near zero
Z for Age=25: (25 - 58.3) / 64.8 ≈ -0.51
Z for Age=30: (30 - 58.3) / 64.8 ≈ -0.44
Z for Age=200: (200 - 58.3) / 64.8 ≈ +2.19
```

Python

The ages 25–35, representing the true bulk of the data, all get squashed into a narrow band near 0. The model loses the ability to distinguish between them.

8.2 Solution — RobustScaler

- RobustScaler uses the median and Interquartile Range (IQR) instead of mean and std — far less sensitive to outliers.

Python

```
# Import RobustScaler for outlier-heavy data
from sklearn.preprocessing import RobustScaler

# Uses median and IQR instead of mean and std
robust = RobustScaler()
X_scaled = robust.fit_transform(X)
```

Rule of Thumb: • No significant outliers → use StandardScaler • Significant outliers present → use RobustScaler or clip outliers first, then use StandardScaler

9. When NOT to Standardize

- Tree-based models (Decision Tree, Random Forest, XGBoost, LightGBM): No need. These are completely scale-invariant by design.
- Target variable y for classification: y is a label, not a feature — never scale it for classifiers.
- Binary (0/1) features: Scaling is harmless but adds no value and reduces interpretability.
- When magnitude carries meaning: e.g., raw pixel intensities where 255 deliberately means 'bright'. Scaling removes that semantic.
- Sparse data (many zeros): `StandardScaler` shifts values and destroys sparsity. Use `MaxAbsScaler` instead, which preserves zeros.

10. Standardization vs Other Scalers — At a Glance

Scaler	Formula	Properties & Best Use
StandardScaler (Standardization)	$z = (x - \mu) / \sigma$	Mean=0, Std=1. Not bounded. Sensitive to outliers. Best default choice.
MinMaxScaler (Normalization)	$x' = (x - \min) / (\max - \min)$	Range [0,1]. Sensitive to outliers. Good when bounded range is needed.
RobustScaler	$x' = (x - \text{median}) / \text{IQR}$	Robust to outliers. Not bounded. Best when outliers are significant.
MaxAbsScaler	$x' = x / \max(x)$	Range [-1,1]. Preserves sparsity. Good for sparse / NLP data.
Normalizer (row-wise)	Scale each row to unit norm	Per-sample, not per-feature. Used in text / NLP tasks.

11. Common Mistakes to Avoid

- **Fitting scaler on the full dataset before train-test split:** Data leakage. Always split first, then fit on training only.
- **Calling `fit_transform` on test data:** Reversed! `fit_transform` goes on training data; use `transform` on test.
- **Assuming standardization makes data Gaussian:** It only recentres and rescales. The shape is completely preserved.
- **Scaling the target y for classification:** y is a label — never scale it for classifiers.
- **Forgetting inverse `transform` for regression predictions:** If you scaled y for regression, predictions are in z-score space — invert them before reporting results.
- **Using `StandardScaler` when outliers dominate:** Use `RobustScaler` or clip outliers first.

12. Summary — Everything in One Place

Concept	Key Takeaway
Why scale?	Different feature ranges bias distance/gradient algorithms unfairly
Standardization	Transform so $\mu=0$, $\sigma=1$ using $z = (x - \mu) / \sigma$
μ and σ learned from	Training set ONLY — never from test or validation set
Shape change?	No — only centre and spread change; distribution shape stays same
Outlier impact	High — outliers inflate σ , compressing all other values
Better for outliers?	RobustScaler (uses median & IQR instead of mean & std)
sklearn class	StandardScaler() in sklearn.preprocessing
fit()	Learns μ and σ from data — call only on training data
transform()	Applies $z = (x - \mu) / \sigma$ using stored statistics
fit_transform()	Does fit() + transform() in one call — training data only
inverse_transform()	Reverses the scaling — useful for interpreting predictions
Best practice	Wrap in sklearn Pipeline to eliminate data leakage risk
Must use for	KNN, SVM, PCA, Regularized Regression, Neural Networks
Not needed for	Decision Trees, Random Forests, XGBoost (tree-based)
Related scalers	MinMaxScaler, RobustScaler, MaxAbsScaler, Normalizer

Notes by: Amol Jagtap | amoljagtap3001@gmail.com | Day 24 — Feature Scaling: Standardization